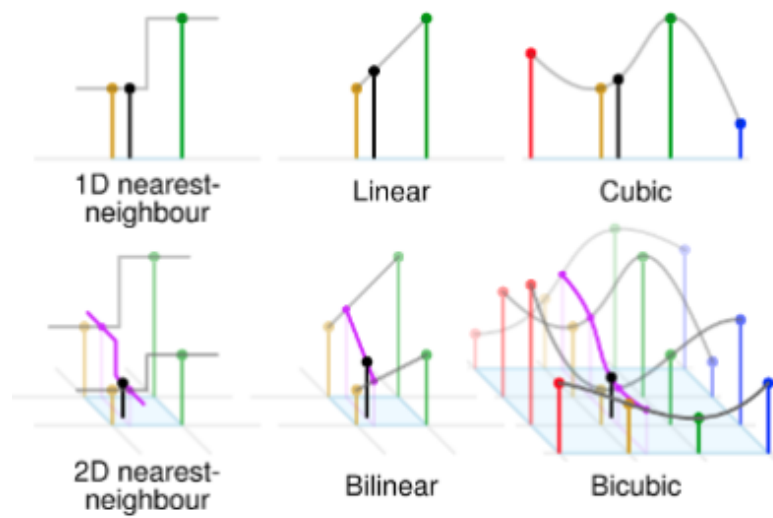


Analyse numérique



Interpolation

Contents

1	Introduction	2
2	Les différents types d'interpolation	2
2.1	Interpolation par la valeur du plus proche voisin	2
2.2	Interpolation linéaire	3
2.3	Interpolation polynomiale	6
2.4	Interpolation polynomiale	6
2.4.1	Interpolation de Lagrange	6
2.4.2	Estimation de l'erreur d'interpolation	8

1 Introduction

Dans ce document, nous allons parler de l'interpolation mathématique. C'est une des opérations mathématiques utilisées dans les applications suivantes :

- Construction des réseaux de communication
- Redimensionnement des images
- Modélisation et prédiction des paramètres environnementaux sur une grande zone à partir de mesures prises à des endroits discrets
- Estimation et insertion des valeurs inconnues entre des données connues

et bien d'autres encore. Cette opération vise à :

- Remplacer une courbe par une autre courbe plus simple qui coïncide avec la première en un nombre fini de points connus (lissage d'une courbe expérimentale, par exemple)
- Estimer des valeurs de points situés entre des points connus

2 Les différents types d'interpolation

Les types d'interpolation sont les suivants :

Le plus proche voisin : utilise la valeur de la donnée la plus proche.

Linéaire : construit des segments de droites pour relier les points consécutifs.

Polynomiale : ajuste un polynôme à notre ensemble de données.

Spline : construit des polynômes par morceaux assurant une transition fluide aux points de données.

2.1 Interpolation par la valeur du plus proche voisin

L'interpolation par le plus proche voisin consiste à rechercher, pour un point x donné, le point connu x_i le plus proche, puis à affecter à x la valeur de l'ordonnée correspondante y_i .

Autrement dit, il s'agit d'estimer la valeur de la fonction en un point inconnu en utilisant la valeur de la donnée connue la plus proche. Cela revient à calculer :

$$f(x) = y_i \tag{1}$$

tel que $|x - x_i|$ soit minimal parmi tous les points connus $P_i(x_i, y_i)$.

Cette méthode est simple et rapide à mettre en œuvre, mais elle introduit des discontinuités dans l'interpolation, car la fonction obtenue n'est pas continue.

Voici un exemple de code Python implémentant l'interpolation par le plus proche voisin :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 # -*- coding: utf-8 -*-
4
5 """ By Fabrice PAKA, le code fonctionne!!! """
6
7 def f_plus_proche_voisin(x, xdata, fdata):
8     """Retourne l'interpolation par plus proche voisin d'une fonction au point x.
9         xdata et ydata sont les points de données utilisés pour l'interpolation.
10         On suppose que xdata est trié par ordre croissant"""
11     ind = np.searchsorted(xdata, x) # Search the right interval for point x
12     if (ind == 0):
13         return fdata[0]
14     if (ind == len(xdata)):
15         return fdata[-1]
16     x0,f0 = xdata[ind-1],fdata[ind-1]
17     x1,f1 = xdata[ind],fdata[ind]
18     if (abs(x-x0) < abs(x-x1)):
19         return f0
20     else:
21         return f1
22
23 if __name__=='__main__':
24     # Calcul des valeurs de f(x) en utilisant l'interpolation lineaire
25     # Les données d'interpolation.
26     xdat = np.array([0, 1, 2, 3, 4, 5, 6])
27     fdat = np.sin(xdat)
28     # Référence pour la fonction réelle
29     xref = np.linspace(0,6,100)
30     fref = np.sin(xref)
31     xcalc = np.linspace(0,6,100)
32     fcalc = [f_plus_proche_voisin(xin,xdat,fdat) for xin in xcalc]
33
34     plt.title("Interpolation par le point le plus proche voisin")
35     plt.xlabel("x")
36     plt.ylabel("f(x)")
37     plt.plot(xdat,fdat,'or',label='data')
38     plt.plot(xcalc,fcalc,label=' Interpolation plus proche voisin')
39     plt.plot(xref,fref,'--',label='La Vraie fonction')
40     plt.legend()
41     plt.savefig("interpolation_plus_proche_voisin.png")
42     plt.show()
```

Voici le résultat obtenu après exécution de ce code :

2.2 Interpolation linéaire

Dans ce cas, la courbe d'interpolation est formée d'une suite de segments reliant chaque paire de points consécutifs. La fonction interpolée est donc une fonction ***affine par morceaux***.

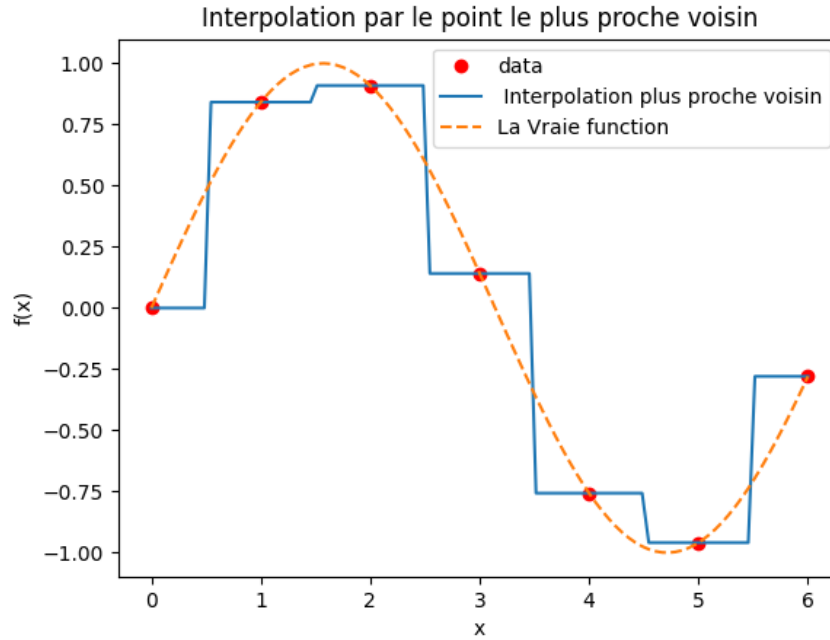


Figure 1: Points connus vs interpolation par le plus proche voisin vs fonction réelle

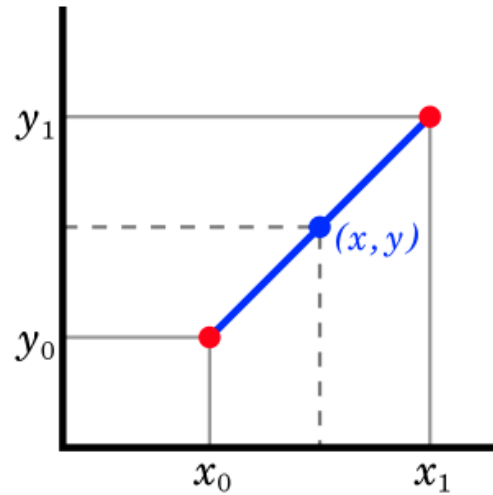


Figure 2: Exemple d'interpolation linéaire entre deux points successifs

La formule utilisée est la suivante :

$$y = p(x - x_0) + y_0 \quad (2)$$

où $p = \frac{y_1 - y_0}{x_1 - x_0}$ est le **coefficient directeur** de la droite passant par les deux points $P_0(x_0, y_0)$ et $P_1(x_1, y_1)$.

Le point (x, y) est une estimation linéaire située entre P_0 et P_1 .

Voici un exemple de code Python permettant d'effectuer une interpolation linéaire :

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 # -*- coding: utf-8 -*-
4
5 """ By Fabrice PAKA, le code fonctionne!! """
6
7 def interpolation_lineair(x,x0,f0,x1,f1):
8     """Retourne la valeur d'une fonction a un point x
9     à travers interpolation lineaire entre les points (x0,y0) et (x1,y1)."""
10    return f0 + (f1 - f0) * (x-x0) / (x1-x0)
11
12
13 def f_interpoler_lineairement(x, xdata, fdata):
14     """Retourne l'interpolation lineaire d'une fonction en un point x.
15     xdata et ydata sont les points utilisé pour faire l'interpolation.
16     xdata est supposé être iordonné de façon croissante."""
17    ind = np.searchsorted(xdata, x) # Cherche le bon intervalle pour x
18    if (ind == 0):
19        #if ((xdata[0] - x) > 1e-12):
20        if (xdata[0] > x):
21            print(f"x = { x} est hors de la l'intervalle d'interpolation [{xdata[0]}, {xdata[-1]} ]")
22            ind = ind + 1
23    if (ind == len(xdata)):
24        if (x > xdata[-1]):
25            #if ((x - xdata[-1]) > 1e-12):
26            print(f"x = { x} est hors de la l'intervalle d'interpolation [{xdata[0]}, {xdata[-1]} ]")
27            ind = ind - 1
28    x0,f0 = xdata[ind-1],fdata[ind-1]
29    x1,f1 = xdata[ind],fdata[ind]
30    return interpolation_lineair(x,x0,f0,x1,f1)
31
32 if __name__ == "__main__":
33     print("This script is not meant to be run directly.")
34     # Calcul des valeurs de f(x) en utilisant l'interpolation lineaire
35     # Les données d'interpolation.
36     xdat = np.array([0, 1, 2, 3, 4, 5, 6])
37     fdat = np.sin(xdat)
38     # Référence pour la fonction réelle
39     xref = np.linspace(0,6,100)
40     fref = np.sin(xref)
41
42     # Interpolation lineaire
43     xcalc = np.linspace(0,6,100)
44     fcalc = [f_interpoler_lineairement(xin, xdat, fdat) for xin in xcalc]
45
46     # Affichage des résultats
47     plt.title("Linear Interpolation: True Function vs Approximation")
48     plt.xlabel("x")
49     plt.ylabel("f(x)")
50     plt.plot(xdat,fdat,'or',label="Données d'interpolation")
51     plt.plot(xcalc,fcalc,label='Interpolation lineaire')
52     plt.plot(xref,fref,'--',label=' La vraie fonction')
53     plt.legend()
54     plt.savefig('interpolation_plot.png')
55     plt.show()

```

Voici le résultat obtenu après exécution de ce code:

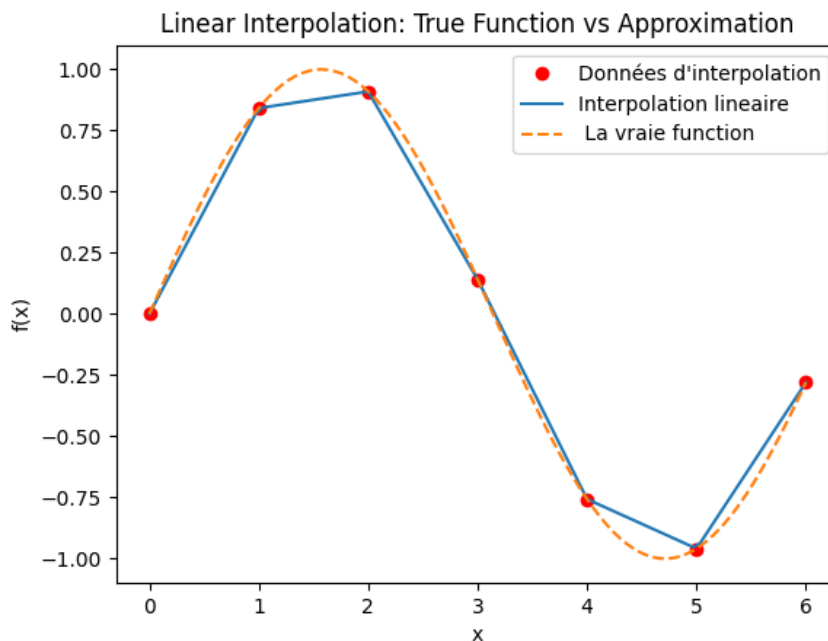


Figure 3: Points connus vs interpolation linéaire vs la vraie fonction

2.3 Interpolation polynomiale

2.4 Interpolation polynomiale

Dans ce cas, la fonction est approchée par un polynôme.

Si l'on dispose de $n+1$ points distincts $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, alors il existe un **unique polynôme** de degré au plus n qui passe exactement par chacun de ces points.

Il existe plusieurs méthodes pour effectuer une interpolation polynomiale. Ici, nous allons nous concentrer sur l'interpolation de **Lagrange**.

2.4.1 Interpolation de Lagrange

L'interpolation de Lagrange repose sur la construction de **polynômes de base de Lagrange**, notés $L_i(x)$, définis par :

$$L_i(x) = \prod_{0 \leq j \leq n, j \neq i} \frac{x - x_j}{x_i - x_j} \quad (3)$$

Chaque polynôme $L_i(x)$ vérifie la propriété suivante :

$$L_i(x_j) = \begin{cases} 1 & \text{si } j = i \\ 0 & \text{si } j \neq i \end{cases}$$

Ainsi, le polynôme d'interpolation $P_n(x)$ s'écrit comme une combinaison linéaire de ces fonctions de base pondérées par les valeurs y_i :

$$P_n(x) = \sum_{i=0}^n y_i \cdot L_i(x) \quad (4)$$

Ce polynôme passe exactement par tous les points donnés.

Voici un exemple de code Python permettant de faire une interpolation de Lagrange :

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  # -*- coding: utf-8 -*-
4
5  """ By Fabrice PAKA, le code fonctionne !!! """
6
7  def Lkj(x,j,xdata):
8      """Fonction de base de lagrange."""
9      ret = 1.
10     #for k in range(0, len(xdata)):
11     #     if (k != j):
12     #         ret *= (x - xdata[k]) / (xdata[j] - xdata[k])
13     ret *= np.prod([(x - xdata[k]) / (xdata[j] - xdata[k]) for k in range(len(xdata)) if k != j])
14     return ret
15
16  def f_interpolation_polynomial(x, xdata, fdata):
17      """Retourne l'interpolation polynomiale d'une fonction au point x.
18      xdata et ydata sont les points de données utilisés pour l'interpolation."""
19      ret = 0.
20      n = len(xdata) - 1
21      for j in range(0, n+1):
22          ret += fdata[j] * Lkj(x,j,xdata)
23      return ret
24
25  if __name__ == "__main__":
26      # Calcul des valeurs de f(x) en utilisant l'interpolation lineaire
27      # Les données d'interpolation.
28      xdat = np.array([0, 1, 2, 3, 4, 5, 6])
29      fdat = np.sin(xdat)
30      # Référence pour la fonction réelle
31      xref = np.linspace(0,6,100)
32      fref = np.sin(xref)
33      # Interpolation polynomiale
34      xpoly = np.linspace(0,6,100)
35      fpoly = [f_interpolation_polynomial(xin,xdat,fdat) for xin in xpoly]
36
37      # Affichage des résultats
38      plt.title("Interpolation polynomiale")
39      plt.xlabel("x")
40      plt.ylabel("f(x)")
41      plt.plot(xdat,fdat,'or',label="Données d'interpolation")
42      plt.plot(xpoly,fpoly,label='Interpolation polynomiale')
43      plt.plot(xref,fref,'--',label='La vraie fonction')
44      plt.legend()

```

```

44 plt.savefig("interpolation_polynomial.png")
45 plt.show()

```

Résultats observés:

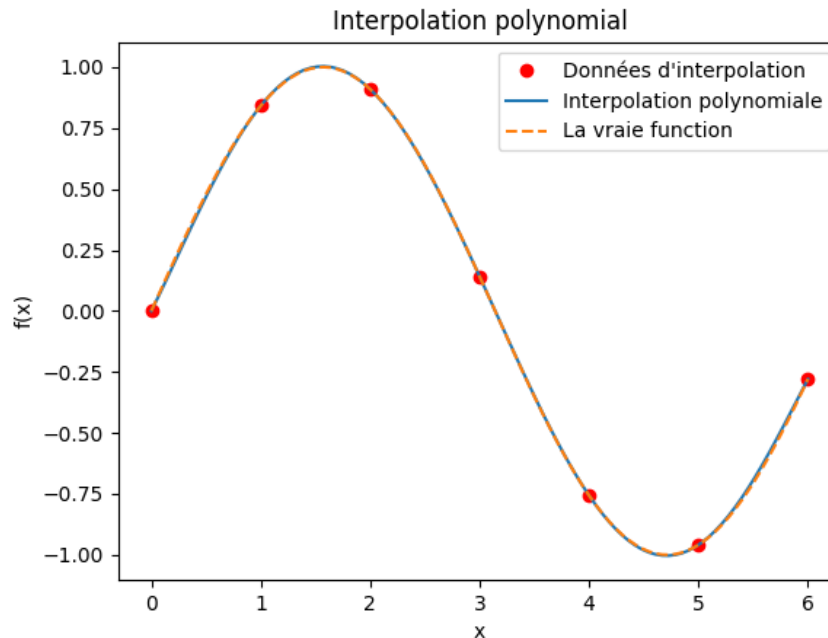


Figure 4: Points connus vs interpolation polynomiale vs la vraie fonction

2.4.2 Estimation de l'erreur d'interpolation

L'erreur de l'interpolation polynomiale est liée à la dérivée d'ordre $(n+1)$ de la fonction, ainsi qu'à la distribution des nœuds.

$$R_n(x) = \frac{f^{(n+1)}}{n+1} \prod_{i=0}^n (x - x_i) \quad (5)$$

Le terme d'erreur dépend de la dérivée d'ordre $(n+1)$ de la fonction. Pour les fonctions ayant de fortes dérivées ou des variations rapides, l'interpolation polynomiale peut devenir imprécise, en particulier avec des nœuds équidistants.

```

1 plt.title("Erreur d'interpolation polynomiale")
2 plt.xlabel("x")
3 plt.ylabel("$|p(x) - \sin(x)|$")
4 plt.plot(xref,abs(fref-fpoly),'--',label='erreur')
5 n = len(xdat) - 1
6 plt.axhline(y = (xdat[1]-xdat[0])**n/4./(n+1.), color = 'r', linestyle = '-',label='erreur maximale')
7 plt.legend()
8 plt.show()

```

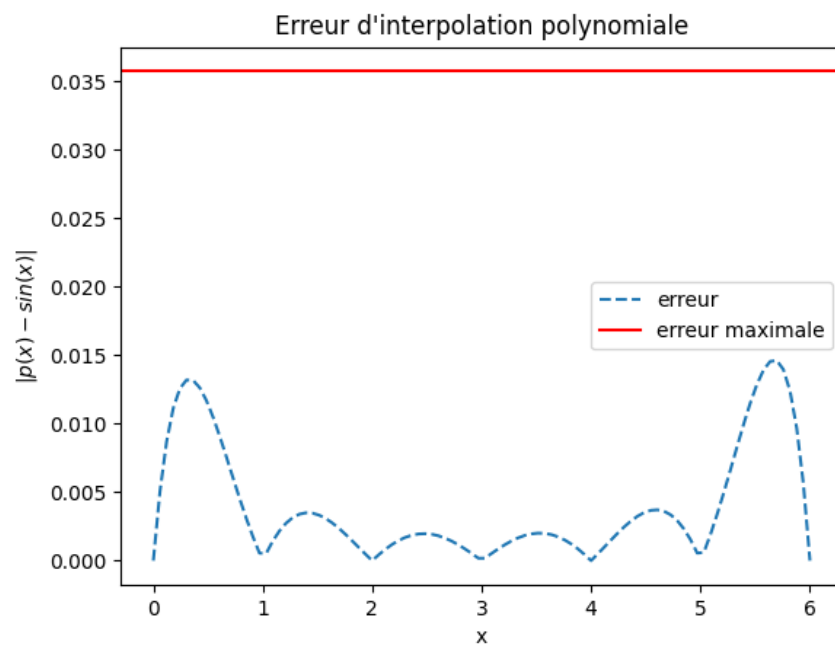


Figure 5: Erreur d'interpolation polynomial de Lagrange